

Dependency Injection

a simple concept behind a fancy term



in object-oriented languages, classes and objects
are the foundations of any functionality

relationships like composition and
inheritance make it possible to extend
and reuse some functionalities

how we build those dependencies, determine
how decoupled and reusable the code will be



what if we need to support more food types?

imagine our business has grown and we
now want to prepare pizzas as well

with our current design, where the Chef creates its own
dependency, we're stuck with three flawed approaches:

1. Replace Burger with Pizza entirely (never make burgers again)
2. Duplicate the prepare method for each food type
3. Create multiple Chef subclasses, one per food type



```
public interface Food { }
```

```
public class Burger implements Food { }
```

```
public class Pizza implements Food { }
```

```
public class Chef {  
    private Food pizza;  
  
    public Chef() {  
        pizza = new Pizza();  
    }  
  
    public void prepareFood() {  
        // do something with  
        // the pizza object  
    }  
}
```

1

the Chef creates its
own dependency
locked to one food type

```
public class Chef {  
    private Food burger;  
  
    public Chef() {  
        burger = new Burger();  
    }  
  
    public void prepareFood() {  
        // do something with  
        // the burger object  
    }  
}
```



```
public interface Food { }
```

```
public class Burger implements Food { }
```

```
public class Pizza implements Food { }
```

```
public class Chef {  
  
    private Food pizza;  
  
    public Chef() {  
        pizza = new Pizza();  
    }  
  
    public void prepareFood() {  
        // do something with  
        // the pizza object  
    }  
}
```

```
public class Chef {
```

```
    private Food burger;  
    private Food pizza;
```

```
    public Chef() {  
        burger = new Burger();  
        pizza = new Pizza();  
    }
```

```
    public void prepareBurger() {  
        // do something with  
        // the burger object  
    }
```

```
    public void preparePizza() {  
        // do something with  
        // the pizza object  
    }
```

```
}
```

**tight coupling → code duplication
for every new food type**



```
public interface Food { }
```

```
public class Burger implements Food { }
```

```
public class Pizza implements Food { }
```

```
public class PizzaChef {  
    private Food pizza;  
  
    public PizzaChef() {  
        pizza = new Pizza();  
    }  
}
```

```
public class BurgerChef {  
    private Food burger;  
  
    public BurgerChef() {  
        burger = new Burger();  
    }  
}
```

```
public class Chef {
```

```
    private Food burger;  
    private Food pizza;
```

```
    public Chef() {  
        burger = new Burger();  
        pizza = new Pizza();  
    }
```

```
    public void prepareBurger() {  
        // do something with  
        // the burger object  
    }
```

```
    public void preparePizza() {  
        // do something with  
        // the pizza object  
    }
```

```
}
```

2

3



```
public interface Food { }
```

```
public class Burger implements Food { }
```

```
public class Pizza implements Food { }
```

```
Chef pizzaChef = new Chef(new Pizza());
```

```
Chef brgrChef = new Chef(new Burger());
```

```
public class Chef {  
  
    private Food food;  
  
    public Chef(Food food) {  
        this.food = food;  
    }  
  
    public void prepareFood() {  
        // do something with  
        // the food object  
    }  
  
}
```

Dependency Injection

decouples the main class
from its dependencies



Before

```
public class Chef {  
  
    private Food burger;  
  
    public Chef() {  
        burger = new Burger();  
    }  
  
    public void prepareFood() {  
        // do something with  
        // the burger object  
    }  
  
}
```

hard-dependency

it cannot be used and
tested independently
from this class



After

```
public class Chef {  
  
    private Food food;  
  
    public Chef(Food food) {  
        this.food = food;  
    }  
  
    public void prepareFood() {  
        // do something with  
        // the food object  
    }  
  
}
```

the dependency is
provided from outside

the dependent class
doesn't have to figure
out what is changing



Types of Dependency Injection

Constructor Injection

Setter Injection

Field Injection



Constructor Injection

```
public class ClassA {  
    private ClassB classB;  
  
    public ClassA(ClassB classB) {  
        this.classB = classB;  
    }  
}
```

the most
recommended
type

```
ClassA classA = new ClassA(new ClassB());
```



Setter Injection

```
public class ClassA {  
    private ClassB classB;  
  
    public ClassA() { }  
  
    public void setClassB(ClassB classB) {  
        this.classB = classB;  
    }  
}
```

```
ClassA classA = new ClassA();  
classA.setClassB(new ClassB());
```

this approach is not
recommended as it
hides the
dependencies

can cause runtime errors
like NullPointerException



Field Injection

```
public class ClassA {  
    public ClassB classB;  
    public ClassA() { }  
}
```

```
ClassA classA = new ClassA();  
classA.classB = new ClassB();
```

same problems as
Setter Injection and it
adds complexity

requires reflection to
modify private fields



now that we know what Dependency Injection is
and the three ways to use it...

let's talk about the principle behind it
why we use it, and when we shouldn't



Inversion of Control

a principle that transfers the control of objects to a framework

a framework takes control over the flow of the program and makes calls to custom code

frameworks rely on abstractions, we extend their classes or plug in our own



Inversion of Control

the 5th principle of SOLID states that a class should depend on abstractions and not on concretions

```
public class Chef {  
  
    private Food food;  
  
    public Chef(Food food) {  
        this.food = food;  
    }  
  
}
```

**depends on the
abstraction**

```
public class Chef {  
  
    private Food burger;  
  
    public Chef() {  
        burger = new Burger();  
    }  
  
}
```

**depends on the
implementation**



Dependency Injection

helps a class focus on fulfilling its responsibilities

reduces boilerplate and duplication

makes it easier to extend functionalities
(open/closed principle)

testing becomes simpler too!



Dependency Injection is a solution, not a default

sometimes injecting dependencies
can overcomplicate things

especially when a dependency is an implementation
detail that should stay encapsulated within its class



don't use it if you don't
have a problem

adding abstraction where none is
needed makes code harder to follow

only introduce DI when the
coupling is causing real pain



when Class B is only used by Class A

if B is fully encapsulated inside A and is just an implementation detail, injecting B into A exposes it unnecessarily

this creates tighter coupling elsewhere in the system
and leaks internal concerns to the outside



Leaking Classes

```
public class ClassA {  
  
    public ClassA() {  
    }  
  
    public void doStuff() {  
        // create classB and  
        // do something with it  
        // clients don't care about it  
        // it's an implementation detail  
    }  
  
}
```

```
public class ClassA {  
  
    private ClassB classB;  
  
    public ClassA(ClassB classB) {  
        this.classB = classB;  
    }  
  
    public void doStuff() {  
        // do something with classB  
    }  
  
}
```

**now B exists in some
other place as well**



Mocking

```
public class ClassA {  
  
    private ClassB classB;  
  
    public ClassA(ClassB classB) {  
        this.classB = classB;  
    }  
  
    public void doStuff() {  
        // do something with classB  
    }  
  
}
```

```
new ClassA(mock(new ClassB()));
```

does the actual
production code of A
work with the actual
production code of B?

passing tests with mocks
≠ production correctness



use dependency injection when it makes sense

inject dependencies to decouple, test, and extend
but keep it simple when the coupling isn't a problem

